

ARK-449 Preregistration

State Changes After Verification

Series: ExecutionProof ARK Authorization-Boundary Track

Version: v1.1 → locked as `ark-449-v1.1-lock` immediately before hardware submission

Repository: <https://github.com/derekhone/executionproof-testbeds>

Folder: `ark-449/`

Target tags: `ark-449-v1.1-lock` (preregistration LOCK) → `ark-449-v1.1` (execution result)

v1.1 PRE-DATA CORRECTION NOTICE. This version supersedes the v1.0 lock **before any hardware data was read** (no principal or SPAM job was ever submitted under v1.0). Two construction defects, caught by the mandatory noiseless dry-run, are corrected here per the ARK-444 pre-data-correction precedent:

1. **ALLOW execution gate** `H` → `X`. v1.0 applied a Hadamard on execution, putting the payload in a 50/50 superposition ($P(Q_P=1) \approx 0.50$). Criterion C1 requires $S_A_{\min} \geq 0.90$, so v1.0 was a **guaranteed FAIL by construction**. The execution gate now drives the payload to $|1\rangle$ via `X`, giving $P(Q_P=1) \approx 1.00$ on a clean device, consistent with the ALLOW/DENY semantics used throughout the corpus and with §14 point 6.
2. **Payload-bit parsing** `split()[-1]` → `split()[0]`. Registers are declared (`c_auth`, `c_state`, `c_pay`); Qiskit prints them reverse-declaration order as “`c_pay c_state c_auth`”, so the payload is the leftmost field. v1.0 read the rightmost field (`c_auth`, always 1). Both the analysis script and the dry-run are corrected.

Post-correction noiseless dry-run: ALLOW arms = 1.0000, all seven DENY arms = 0.0000, $\Delta_B = 1.00$, SPAM gate passed. No criteria, hypotheses, arms, or thresholds were changed. This is a technical correction to make the circuit and parser faithfully implement the already-locked criteria.

Prepared for: Derek Hone, Remnant Fieldworks Inc.

Date drafted: 2026-07-17

Lock date: To be set at moment of hardware submission

LOCK INSTRUCTION

Before any hardware job is submitted, the following steps must be completed in strict order:

1. Finalize this document. No further changes after step 2.
2. Compute SHA-256 hash of this file and all circuit code files.
3. Commit the MANIFEST (see Section 16) to `executionproof-testbeds`.
4. Push and tag the commit `ark-449-v1.0-lock`.
5. Submit the SPAM gate job. Record the job ID in the execution log **before any results are read**.
6. Submit the principal job. Record the job ID in the execution log **before reading SPAM results**.
7. Read results only after both job IDs are committed.

No criterion may be changed, loosened, or added after step 2. A FAIL under these criteria is a valid, publishable result. An ABORT at the SPAM gate is a valid, publishable result. The value of the protocol is that it constrains these choices before any data is seen.

1. Preamble and Series Context

1.1 Where This Fits

The ARK series through ARK-448 established the ExecutionProof authorization boundary across seven distinct dimensions:

Experiment	Dimension Established
ARK-441 / ARK-446	Boundary exists and replicates across devices
ARK-442	Temporal: stale authority fails closed
ARK-444	Integrity: altered decision record fails closed
ARK-443	Multi-party: single compromised authorizer cannot bypass quorum
ARK-445 / ARK-445b	Three-state: HOLD is a metrologically separable first-class outcome
ARK-447	Noise: Pauli twirling provides modest, measurable improvement
ARK-448	Protocol: gate-stop halts before spending budget on marginal conditions

What the corpus has not yet tested: **whether an authorization that was valid at approval time remains valid when the world has changed before execution.**

This is the gap ARK-449 closes.

1.2 The Doctrine Being Tested

The central ExecutionProof claim is:

Permission at approval time is not permission at execution time.

An agent that verifies once and executes at an arbitrary later time treats the approval decision as a permanent pass. ExecutionProof holds that this is a categorical error: execution must be gated on the admissibility of current state, not merely the existence of a prior approval.

ARK-449 is the direct experimental test of that doctrine. It is not a noise study. It is not a hardware characterization. It is a test of whether the complete control model survives the condition that produces the largest class of real-world AI governance failures.

1.3 Precise Differentiation from ARK-444

ARK-444 tested **decision-record integrity**: the approved action was altered before execution. The decision record was tampered with. The world state was consistent.

ARK-449 tests **temporal state validity**: the approved action is unchanged, the decision record is intact, and the original authorization was legitimate at time T_1 . The question is whether that approval remains valid at time T_3 when conditions changed at T_2 .

Dimension	ARK-444	ARK-449
Decision record	Tampered	Intact and legitimate
Approved action	Changed	Unchanged
World state	Consistent	Changed between T_1 and T_3
Question	"Was the action altered?"	"Is the approval still current?"
Doctrine tested	Authorization integrity	Authorization currency

These are different questions with different commercial implications. ARK-444 demonstrated that authorization cannot be detached from the specific approved action. ARK-449 demonstrates that authorization cannot be detached from the moment in time at which state was admissible.

2. Primary Hypothesis

H1 (Primary): When an action is verified against state S_1 at time T_1 , and the relevant world state changes to S_2 (inadmissible) at T_2 before execution at T_3 , the system will DENY execution — and will ALLOW execution only when re-verified against currently admissible state at T_3 .

Operationally:

- **ALLOW condition:** $c_{\text{auth}} = 1 \text{ AND } c_{\text{state}} = 1 \rightarrow \text{execution gate applied} \rightarrow S_A \geq 0.90$
- **DENY condition:** $c_{\text{auth}} = 1 \text{ AND } c_{\text{state}} = 0 \rightarrow \text{no execution gate} \rightarrow L_D \leq 0.02$

H1 is confirmed if and only if **all four primary pass criteria** in Section 9 are met simultaneously.

3. Secondary Hypotheses

H2a — State-change universality: The type of state change is irrelevant to the DENY outcome. Whether authority is revoked, policy changes, account balance falls below threshold, risk score exceeds a limit, destination is blocked, or supporting evidence expires — every category of state change produces $L_D \leq 0.02$. No individual state-change category leaks above threshold.

H2b — Re-authorization restores execution: A new decision issued against a re-assessed admissible state produces ALLOW ($S_A \geq 0.90$). The system is not a one-way gate that permanently blocks an action once any state change has occurred. Legitimate re-authorization under current conditions must restore execution capability.

H2c — Replay attack with changed state fails closed: Presenting the original ProofRecord (old authorization, $c_auth = 1$ from T_1) against a changed inadmissible state ($c_state = 0$ at T_3) produces DENY ($L_D \leq 0.02$). Old approval cannot substitute for current-state verification. This is the operationalization of the core doctrine.

H2d — Boundary separation: The gap between minimum ALLOW retention and maximum DENY leakage ($\Delta_B = S_A_min - L_D_max$) is ≥ 0.70 , confirming the two outcomes are metrologically separable with a margin large enough to be commercially meaningful.

4. Circuit Architecture

4.1 Qubits

- **Q_A** — Authorizer qubit. Encodes the authorization decision at T_1 .
- **Q_P** — Payload qubit. Execution target. An **X** gate drives it to $|1\rangle$ if and only if $c_exec = 1$. (v1.1: v1.0 specified an **H** gate here — see the pre-data correction notice at the top.)

Two qubits total. No inter-qubit two-qubit gates required. All conditioning is classical feedforward, consistent with ARK-443 and ARK-445b precedent.

4.2 Classical State Model

The world state (c_state) is a **classical constant** fixed per arm in the circuit code before lock. It is not measured from a qubit. It is not adjusted after any data is read.

This models the boundary condition: a state that is definitively different at execution time. This design choice keeps the circuit clean and the result interpretable. The distributed-systems complexity of propagating state changes across asynchronous nodes is honestly out of scope — see Section 14.

4.3 Execution Logic

```

Step 1 – Authorization ( $T_1$ ):
  Prepare Q_A per arm specification
  Measure Q_A → c_auth [classical bit]

Step 2 – State assessment ( $T_2 \rightarrow T_3$  gap, modeled classically):
  c_state ← arm-specific constant [0 = inadmissible, 1 = admissible]

Step 3 – Re-verification gate:
  c_exec ← c_auth AND c_state [classical AND]

Step 4 – Execution:
  If c_exec == 1:
    Apply X gate to Q_P [execution – payload driven to  $|1\rangle$ ]
  Else:
    Q_P remains in  $|0\rangle$  [no execution]

Step 5 – Measurement:
  Measure Q_P → outcome bit

```

4.4 Outcome Interpretation

Q_P Measurement Outcome	Interpretation
$P(Q_P = 1) \approx 1.00$	ALLOW — execution gate applied; payload driven to
$P(Q_P = 1) \approx 0.00$	DENY — execution gate not applied; payload in ground state

- **ALLOW retention:** $S_A = P(Q_P = 1)$ for ALLOW arms. Target: ≥ 0.90
- **DENY leakage:** $L_D = P(Q_P = 1)$ for DENY arms. Target: ≤ 0.02

5. Arm Specifications

Nine arms. 8192 shots per arm. All nine submitted in a single principal job.

Arm	Label	Q_A Prep	c_auth	c_state	c_exec	State- Change Scenario	Expected Out- come	Criter- ia
1	ALLOW-un- change d	1⟩	1	1	1	No change — state valid at T_1 and remains valid at T_3	$S_A \geq 0.90$	H1, H2d
2	DENY-revoked	1⟩	1	0	0	Author- ity re- voked between an ap- proval (T_1) and execu- tion (T_3)	$L_D \leq 0.02$	H1, H2a
3	DENY-policy	1⟩	1	0	0	Policy version up- dated; action now in- admiss- ible un- der new policy	$L_D \leq 0.02$	H1, H2a
4	DENY-balance	1⟩	1	0	0	Account balance fell be- low re- quired threshol d after approv- al	$L_D \leq 0.02$	H1, H2a
5	DENY-risk	1⟩	1	0	0	Risk score crossed	$L_D \leq 0.02$	H1, H2a

Arm	Label	Q_A Prep	c_auth	c_state	c_exec	State- Change Scenario	Expected Outcome	Criteria
						enforcement limit after approval		
6	DENY- destination	1⟩	1	0	0	Destination entity blocked or sanctioned after approval	$L_D \leq 0.02$	H1, H2a
7	DENY- expiry	1⟩	1	0	0	Supporting evidence expired after approval was granted	$L_D \leq 0.02$	H1, H2a
8	ALLOW- reauth	1⟩	1	1	1	New decision issued against re-assessed admissible state; fresh authorization	$S_A \geq 0.90$	H2b
9	DENY- replay	1⟩	1	0	0	Old ProofRecord	$L_D \leq 0.02$	H2c

Arm	Label	Q_A Prep	c_auth	c_state	c_exec	State- Change Scenario	Expected Outcome	Criteria
						presented without re-verification; state changed since original approval		

5.1 Note on Circuit-Equivalent Arms and Scientific Value

Arms 2 through 7 and Arm 9 are circuit-equivalent at the hardware level: $c_{\text{auth}} = 1$, $c_{\text{state}} = 0$, $c_{\text{exec}} = 0$. They are semantically distinct — each models a different real-world class of state invalidation that an enterprise buyer would immediately recognize.

This is consistent with ARK-444's approach (multiple semantically distinct DENY paths — altered destination, altered amount, altered op-type — under the same circuit logic). The scientific value is two-fold:

1. **Universality claim:** The control logic produces fail-closed behavior across every tested category of state change, not just one abstract DENY. H2a requires that $L_D \leq 0.02$ holds for each arm individually — if one category leaks while others do not, that is a finding, not a clean pass.
2. **Commercial clarity:** Enterprise buyers understand authority revocation, policy changes, balance thresholds, and risk scores. Showing that each is handled correctly by the same underlying control structure is the commercially legible form of the claim.

Similarly, Arms 1 and 8 are circuit-equivalent ($c_{\text{auth}} = 1$, $c_{\text{state}} = 1$, $c_{\text{exec}} = 1$). Arm 1 models original approval with unchanged state. Arm 8 models fresh authorization issued under new state after a prior state change. The semantic distinction validates H2b — the system can restore execution via legitimate re-authorization, demonstrating that the control logic is not a one-way block.

6. SPAM Gate

The SPAM gate runs before the principal job is submitted. If the SPAM gate fails, the principal job is **not submitted**. The experiment is recorded as **ABORTED AT SPAM GATE**. No threshold loosening, no re-running until it passes, no proceeding on a failed gate. These prohibitions are absolute under the lock.

6.1 SPAM Job Specification

- **Shots:** 2048
- **SPAM_A circuit:** Prepare Q_A in $|1\rangle \rightarrow$ measure $\rightarrow$ $SPAM_A = P(Q_A = 0)$ [bit-flip readout error on authorizer qubit]
- **SPAM_P circuit:** Prepare Q_P in $|+\rangle$ via H gate $\rightarrow$ measure $\rightarrow$ $SPAM_P = |P(Q_P = 1) - 0.5|$ [symmetry deviation of payload qubit]

6.2 SPAM Pass Criteria

Both checks must pass. Either failing $\rightarrow$ abort.

Check	Threshold	Pass Condition
SPAM_A	≤ 0.02	Authorizer readout error within tolerance
SPAM_P	≤ 0.02	Payload

6.3 SPAM_P Role

SPAM_P is a **gating diagnostic only**. Per the ARK-447 v1.1 correction, SPAM_P is not subtracted from DENY leakage. $SPAM_P \approx 0.5$ represents the expected readout of a $|+\rangle$ superposition — subtracting it from L_D would artificially force leakage to zero and invalidate the measurement. SPAM_A is applied as a readout correction to ALLOW arm results only, where it represents genuine bias. This distinction is fixed at lock and cannot be changed post-data.

7. Qubit Selection Rule

Select the connected pair (Q_A , Q_P) with the lowest sum of single-qubit readout errors from the calibration snapshot taken at submission time. Both individual readout errors must be ≤ 0.02 . If no connected pair satisfies both constraints, select the lowest-sum connected pair available and record the deviation explicitly.

Because all conditioning is classical feedforward (no inter-qubit two-qubit gates required), there is no connectivity constraint beyond what the backend imposes on mid-circuit measurement and feedforward. Select for readout quality, not gate fidelity.

The same qubit pair is used for both the SPAM gate job and the principal job. Qubit selection is fixed at the time of SPAM gate submission and is not changed between jobs.

Record at selection time: Q_A index, Q_P index, individual readout errors, sum_RE, calibration snapshot timestamp. Commit before SPAM gate submission.

8. Shot Counts and Execution Plan

Job	Circuits	Shots/Circuit	Total Shots
SPAM gate	2 ($Q_A 1\rangle, Q_P +\rangle$)	2048	4096
Principal	9 arms	8192	73,728

Principal job submitted only if SPAM gate passes.

8.1 Strict Execution Sequence

All steps must be executed in order. Commit timestamps must prove the ordering.

1. Commit MANIFEST (SHA-256 hashes of this document + all circuit files)
→ push → tag ark-449-v1.0-lock
2. Run qubit selection per Section 7
→ record Q_A , Q_P , readout errors in execution log
→ commit execution log entry
3. Submit SPAM gate job
→ record SPAM job ID in execution log
→ commit before reading any results
4. Submit principal job
→ record principal job ID in execution log
→ commit before reading SPAM results
5. Read SPAM gate results
→ apply gate decision per Section 6.2
- 6a. If SPAM gate PASSED → read principal results → apply analysis per Section 11
- 6b. If SPAM gate FAILED → record abort → stop. No principal data read or claimed.

9. Pass / Fail Criteria (Primary — Preregistered)

All four criteria must be satisfied simultaneously for a PASS verdict. Failure of any single criterion = FAIL verdict for the entire experiment, regardless of how other criteria performed.

#	Criterion	Threshold	Arms
C1	S_A_min — minimum ALLOW retention across ALLOW arms	≥ 0.90	Arms 1, 8
C2	L_D_max — maximum DENY leakage across all DENY arms	≤ 0.02	Arms 2, 3, 4, 5, 6, 7, 9
C3	$\Delta_B = S_A_{\min} - L_D_{\max}$ — boundary separation	≥ 0.70	Derived
C4	SPAM gate — both SPAM_A ≤ 0.02 and SPAM_P ≤ 0.02	≤ 0.02 each	Pre-principal

9.1 Criterion Independence

C1 and C2 are evaluated on raw counts, with SPAM_A correction applied to ALLOW arm results only (if SPAM_A > 0). C3 is derived from C1 and C2 and is not independently measured. C4 is evaluated before the principal job is submitted; a C4 failure terminates the experiment before C1–C3 can be evaluated.

10. Secondary Metrics (Descriptive — Not Pass/Fail)

Reported regardless of primary verdict. These provide diagnostic depth and inform the forward roadmap. They do not alter the pass/fail outcome.

Metric	Description	Hypothesis
S_A per arm	ALLOW retention for Arms 1 and 8 individually	H2b
L_D per arm	DENY leakage for each of Arms 2–7, 9 individually	H2a
L_D_max by category	Which state-change category (if any) shows highest leakage	H2a
L_replay	Leakage for Arm 9 specifically (old-approval replay attack)	H2c
In-situ SPAM drift	$ P(Q_A=0) - \text{SPAM_A_baseline} $ across arms	Readout stability

If any individual DENY arm leaks above 0.02 while L_D_max (averaged or across all arms) remains below, the secondary analysis distinguishes which category of state change caused the breach — a finding worth publishing precisely because it names the hardware or logic constraint.

11. Analysis Plan

11.1 Raw Count Extraction

For each arm, extract:

- `counts_0` — Q_P measured as $|0\rangle$
- `counts_1` — Q_P measured as $|1\rangle$
- $P(Q_P = 1) = \text{counts_1} / (\text{counts_0} + \text{counts_1})$

11.2 SPAM Correction

If $SPAM_A > 0.01$ (non-negligible readout bias on the authorizer qubit), apply the standard single-qubit readout correction to ALLOW arm retention:

$$S_A_corrected = (P(Q_P=1) - SPAM_A) / (1 - 2 * SPAM_A)$$

$SPAM_P$ is used only as a gate diagnostic. It is NOT subtracted from L_D under any circumstances. This rule is fixed at lock per the ARK-447 v1.1 correction and cannot be changed after data is read.

11.3 Primary Metric Computation

```
# ALLOW arms
S_A_arm1 = corrected_P1(arm=1)
S_A_arm8 = corrected_P1(arm=8)
S_A_min  = min(S_A_arm1, S_A_arm8)

# DENY arms
L_D_arms = [P1(arm) for arm in [2, 3, 4, 5, 6, 7, 9]]
L_D_max  = max(L_D_arms)

# Boundary separation
Delta_B = S_A_min - L_D_max
```

11.4 Verdict Assignment

```
if not SPAM_gate_passed:
    VERDICT = "ABORTED AT SPAM GATE"
elif S_A_min >= 0.90 and L_D_max <= 0.02 and Delta_B >= 0.70:
    VERDICT = "PASS"
else:
    VERDICT = "FAIL"
```

11.5 Secondary Analysis

- Report L_D per arm to evaluate H2a (state-change universality — no single category leaks)
- Report S_A per ALLOW arm to evaluate H2b (re-authorization restores execution)
- Report L_replay (Arm 9) explicitly to evaluate H2c (replay with changed state fails closed)
- Report in-situ SPAM drift across arms to characterize readout stability during the run

- If FAIL: identify which criterion failed and which arm(s) contributed — this is the diagnostic for the next experiment

12. What Constitutes Failure (and Why Each Is Still Valuable)

12.1 Any DENY arm leaks above 0.02

A specific class of state change fails to hold the boundary closed. This is a significant, publishable finding: it names the exact category of enterprise state invalidation that the current control logic (or hardware) does not handle. The secondary analysis identifies which arm and at what level, giving the forward program a precise target.

12.2 Any ALLOW arm falls below 0.90

Legitimate re-verified execution is being blocked. This would indicate that the re-verification logic is either too restrictive, or the hardware is performing below the SPAM-corrected tolerance established in earlier ARK experiments. Diagnostic: compare in-situ SPAM drift and calibration snapshot to ARK-441/446 baseline.

12.3 $\Delta_B < 0.70$

The boundary exists in principle but without sufficient separation to be practically meaningful. The two outcome states are not reliably distinguishable at scale. This would not contradict H1 on its own but would substantially limit the claim about practical deployability.

12.4 SPAM gate fails

Readout condition on the selected qubit pair is outside preregistered tolerance at the time of submission. No verdict on state-change behavior is claimed. The experiment is recorded as a preregistered abort. Budget is conserved. A future attempt requires a new locked run — new tag, new SPAM gate, fresh calibration snapshot — not a silent retry of this one.

All four outcomes are valid, publishable results under the preregistration-first protocol. A FAIL that identifies which state-change category leaks is worth more to the program than a marginal pass that obscures the boundary.

13. Pre-Execution Commitments (Lock Rules)

The following are unconditionally prohibited after the MANIFEST is committed (step 1 of Section 8):

1. **No criterion changes.** Thresholds, metric definitions, and the pass/fail verdict logic may not be altered.
2. **No arm additions or removals.** The nine-arm design is fixed. Arms may not be excluded from the primary verdict calculation after data is seen.
3. **No rescue-after-failure.** If the SPAM gate fails: no re-running until it passes, no threshold loosening, no soft abort that proceeds to the principal job anyway.
4. **No post-hoc subgroup selection.** If some DENY arms pass and others fail, the verdict is FAIL — not “partial pass excluding arm N.”

5. **No silent technical corrections after data is read.** Any technical error discovered before data is read (hardware error code, zero-count job, transpilation fault) is documented and corrected in a pre-data correction note — following the ARK-444 precedent. Any correction after data is read requires full explicit disclosure and tagging (following the ARK-447 v1.0→v1.1 precedent).
 6. **Qubit pair is fixed at SPAM gate submission.** No mid-experiment qubit switching based on preliminary SPAM results.
 7. **Analysis code is committed at lock.** The analysis script that produces the verdict runs on the committed raw data without post-hoc modification.
-

14. Honest Boundary Statements

These belong in every external conversation that references ARK-449 results.

1. Classical state model. The world state (`c_state`) is a deterministic classical parameter set per arm, not measured from a live external system. This tests the authorization control logic under the assumption of definitive, known state — not the distributed-systems challenge of propagating state updates across asynchronous nodes with propagation delays, network partitions, or out-of-order delivery. The experiment proves the boundary condition; it does not model the full complexity of real-time state synchronization.

2. Deterministic state change. In production, state changes may be ambiguous, gradual, or probabilistic. This experiment tests the boundary condition where state has definitively changed. Ambiguous or conflicting state signals — where one source says valid and another says revoked — are deferred to ARK-453 (Conflicting Evidence).

3. Timing model. The time elapsed between T_1 (authorization) and T_3 (execution) is modeled logically as sequential circuit arms, not as physical elapsed real-world time. The experiment demonstrates the correctness of the state-check control logic, not the latency characteristics or race conditions of real-world state propagation.

4. Hardware scope. Results apply to the specific backend (`ibm_marrakesh`), qubit pair, calibration snapshot, and shot counts used. Generalization to other hardware, different qubit counts, or different noise regimes requires separate experiments.

5. Not a cryptographic proof. These are hardware noise-characterization studies. The authorization boundary is statistically characterized under a preregistered protocol. This is not a formal security proof and does not claim resistance to adversarial conditions beyond those explicitly tested.

6. ALLOW and DENY are physical outcomes. $S_A \approx 0.97$ does not mean the payload executed with 97% probability in a classical sense — it means the payload qubit was measured in the state consistent with execution with that frequency. The physical outcome encodes the authorization decision; it does not directly execute a financial transaction or API call.

15. Connection to Corpus and Forward Roadmap

15.1 What ARK-449 Adds to the Established Corpus

If ARK-449 passes, the corpus establishes:

Dimension	Established By
Boundary exists and replicates across devices	ARK-441, ARK-446
Temporal: stale authority fails closed	ARK-442
Integrity: tampered action fails closed	ARK-444
Multi-party: single compromised authorizer fails closed	ARK-443
Three-state: HOLD is metrologically separable	ARK-445b
Noise: empirically testable mitigation	ARK-447
Protocol: gate-stop preserves integrity under marginal conditions	ARK-448
State change: authorization currency expires with world state	ARK-449

ARK-449 would be the first experiment in the corpus to directly validate the temporal dimension of the core doctrine at the execution-control level — not merely whether authority was granted at T_1 , but whether conditions at T_3 still warrant it.

15.2 What ARK-449 Enables Downstream

- **ARK-452 (Multi-Step Workflow)** becomes much stronger with ARK-449 in the corpus. A multi-step workflow with one invalid step is precisely a state-change problem at the step level. ARK-449 establishes the single-step boundary; ARK-452 extends it to sequential chains.
- **ARK-453 (Conflicting Evidence Must HOLD)** takes the ambiguous-state-change case that ARK-449 deliberately excludes (see Section 14.2) and tests it directly. The two experiments are complementary by design.
- **ARK-451 (Authority Revocation During Execution)** extends Arm 2 of ARK-449 from a pre-execution state change to a mid-execution revocation. ARK-449 establishes the boundary; ARK-451 tests the boundary under active execution conditions.

15.3 Items Explicitly Deferred from ARK-449

- **HOLD under ambiguous state change.** A state that is probabilistically or partially invalidated — the risk score is near but not at the threshold, or one registry says revoked while another says valid — is deferred to ARK-453. Including it in ARK-449 would require a third qubit (Q_S as a probabilistic state qubit) and would conflate two distinct experimental questions.
 - **Dynamical decoupling.** Remains genuinely open from ARK-448's gate-stop abort. A future locked run when IBM budget refreshes.
 - **Mid-execution state change.** Authority revoked during a multi-step execution sequence (not just before it) is ARK-451.
-

16. MANIFEST Placeholder

Compute and fill at lock time, immediately before hardware submission. Do not fill in advance.

```
ARK-449 MANIFEST – Version 1.0
Lock timestamp: [FILL AT LOCK – UTC, format YYYY-MM-DDTHH:MM:SSZ]
Backend target: ibm_marrakesh (156-qubit Heron r2)

SHA-256 hashes:
  ARK_449_preregistration.md:      [COMPUTE AT LOCK]
  circuits/ark_449_circuit.py:     [COMPUTE AT LOCK]
  circuits/ark_449_analysis.py:    [COMPUTE AT LOCK]

MANIFEST SHA-256 (of this file after all hashes filled):
                                     [COMPUTE AT LOCK]

Committed to:   executionproof-testbeds
Branch:         ark-449/execute (or main, per repository convention)
Tag:            ark-449-v1.0-lock
Commit SHA:     [FILL AFTER PUSH]

SPAM job ID:    [FILL BEFORE READING ANY RESULTS]
Principal job ID:[FILL BEFORE READING SPAM RESULTS]
```

17. ProofRecord Stub

A `proofrecord.json` will be generated at the conclusion of the experiment, following the format established in ARK-448. It will contain:


```

{
  "experiment": "ARK-449",
  "doctrine_tested": "Permission at approval time is not permission at execution
time",
  "verdict": "[PASS | FAIL | ABORTED AT SPAM GATE]",
  "timestamp_lock": "[UTC]",
  "timestamp_execution": "[UTC]",
  "backend": "ibm_marrakesh",
  "qubit_pair": {"Q_A": null, "Q_P": null},
  "spam_gate": {"SPAM_A": null, "SPAM_P": null, "passed": null},
  "primary_metrics": {
    "S_A_arm1": null,
    "S_A_arm8": null,
    "S_A_min": null,
    "L_D_max": null,
    "Delta_B": null
  },
  "secondary_metrics": {
    "L_D_per_arm": {},
    "L_replay_arm9": null
  },
  "criteria": {
    "C1_S_A_min_pass": null,
    "C2_L_D_max_pass": null,
    "C3_Delta_B_pass": null,
    "C4_SPAM_pass": null
  },
  "manifest_sha256": "[FILL AT LOCK]",
  "zenodo_doi": "[FILL AT PUBLICATION]"
}

```

This document is the preregistration for ARK-449. It is locked at the time of MANIFEST commit and tag. No changes to hypotheses, criteria, arms, thresholds, or analysis plan are permitted after that point. A FAIL or ABORT is a valid, publishable result that advances the program. The value of the protocol is precisely that it constrains choices before any data is seen and keeps the record whole regardless of outcome.

Remnant Fieldworks Inc. — ExecutionProof ARK Series
<https://github.com/derekhone/executionproof-testbeds>